

PS8: AI for Industrial Knowledge Intelligence

Comprehensive Hackathon Project Proposal

1. Executive Summary

Problem Statement: Industrial companies store critical knowledge across fragmented sources—maintenance reports, SOPs, OEM manuals, inspection records, compliance documents, and incident logs. Currently, engineers spend 15-25% of their time searching multiple systems and PDFs to answer simple questions like "Why did this pump fail?" or "What's the maintenance schedule?"

Solution: A unified AI-powered knowledge companion that instantly answers equipment-related questions by:

- Searching across all uploaded documents simultaneously
- Providing context-aware answers with exact citations
- Identifying patterns (repeated failures, seasonal issues)
- Predicting failures before they happen
- Recommending preventive actions backed by historical data

Expected Impact:

- 60%+ reduction in documentation search time
 - Early detection of equipment failures (prevent costly downtime)
 - Regulatory compliance automation
 - Institutional knowledge preservation (departing employees)
-

2. Core Features (MVP)

2.1 Document Management

- **Multi-format upload:** PDFs, Word docs, Excel sheets, scanned images
- **Automatic preprocessing:**
 - OCR for scanned documents (Tesseract/EasyOCR)
 - Extract text + metadata
 - Split into semantic chunks (not naive character splits)
 - Preserve document structure (maintain table relationships)
 - Extract equipment metadata (serial numbers, models, installation dates)
- **Metadata tagging:**

- Equipment name/ID (auto-extracted or manual)
- Document type (SOP, Manual, Report, Inspection)
- Date range covered
- Compliance relevance (if applicable)

2.2 AI Search & Retrieval

- **Hybrid search:**
 - Dense vector search (semantic understanding)
 - BM25 lexical search (exact term matching)
 - Combine scores for better recall
- **Equipment-aware retrieval:**
 - Search by equipment name/ID
 - Return: maintenance history, failure logs, SOPs, specifications
 - Filter by date range
- **Smart ranking:**
 - Prioritize recent failures
 - Boost critical alerts
 - Recency decay (older docs less relevant)
 - Relevance + credibility scoring

2.3 AI Chat Assistant

- **Context-aware responses:**
 - User: "Why is Pump A failing?"
 - AI: Searches failure logs, maintenance reports, OEM manuals
 - Response includes:
 - Root cause (with confidence)
 - Similar historical incidents
 - Recommended action
 - Exact citations (document + page)
 - **Follow-up queries:**
 - User: "How often should we replace this bearing?"
 - Maintains conversation context
 - References previous answers
 - **Tone switching:**
 - Technical mode (detailed specs)
 - Manager mode (cost + impact summary)
-

3. Differentiator Features (Stand-Out)

3.1 Root Cause Analysis Engine

Instead of just returning documents, analyze patterns:

User: "Why do we keep replacing pump bearings?"

AI analyzes:

- ✓ All bearing failures in database
- ✓ Operating conditions at time of failure
- ✓ Environmental factors (temperature, humidity)
- ✓ Maintenance intervals followed?
- ✓ OEM recommendations vs. actual practice

Output:

"Bearing failures correlate with:

1. Operating above 80°C (happens 70% of failures)
2. Maintenance interval extended beyond OEM spec (60% of cases)

Recommendation: Install cooling system + enforce stricter intervals
Estimated savings: \$50K/year"

3.2 Temporal Intelligence & Failure Prediction

- **Timeline visualization:**
 - When did this equipment fail?
 - How often? (Trend: increasing/stable/decreasing)
 - Seasonal patterns? (More failures in summer/winter?)
- **Predictive alerts:**
 - "Compressor-7 shows 3 failures in last 12 months. Similar equipment failed on average 8 months after 3 incidents."
 - "Flag for proactive maintenance"
- **Anomaly detection:**
 - "This bearing replacement is unusual—typically lasts 3 years, replaced after 1.5 years"

3.3 Equipment Relationship Mapping

Build a knowledge graph:

Pump A → supplies → Pipeline B → fills → Tank C → drains → Drain Line D



(if Pump A fails, Pipeline B is affected)

Features:

- Impact analysis: "If Pump A fails, which processes stop?"
- Cascade failure detection: "These 3 failures might be related"
- Dependency-aware alerts: "Pump A is critical; scheduled maintenance will stop Pipeline B"

3.4 Multi-Document Correlation

- "Find all incidents where equipment A and B failed in the same week"
- "Compare failure modes across similar equipment models"
- "Maintenance patterns: when multiple similar units are serviced together vs. separately"

3.5 Table & Schematic Intelligence

- Extract structured data from equipment specification tables
- Understand technical drawings (basic shape recognition)
- Comparison queries: "This motor is similar to the one in Warehouse-2. What was its failure history?"

3.6 Cost Impact Analysis

Every recommendation includes:

"Replace bearing now (preventive):

Cost: \$500 (parts + labor)

Downtime: 2 hours

vs.

Wait for failure:

Emergency repair cost: \$3,000

Downtime: 16 hours (plus lost production)

Risk of cascade failure: 40%

ROI: 5.2x savings by acting now"

3.7 Compliance & Regulatory Tracking

- Flag documents with compliance relevance

- Auto-detect violations: "This equipment hasn't been inspected in 14 months. Regulations require annual inspection."
 - Generate compliance reports
-

4. User Interface Strategy

4.1 Main Chat Interface

Left Sidebar:

- Recent equipment
- Saved searches
- Document library
- Alerts/Notifications

Main Panel:

- Chat area
- Response shows:
 - Answer
 - Citation cards (document name + page)
 - Related incidents
 - Recommended action

Right Sidebar:

- Equipment details
- Timeline of incidents
- Maintenance schedule

4.2 Dashboard

Key Metrics:

- Equipment health score (aggregated from failure data)
- Alert dashboard (overdue maintenance, anomalies)
- Most queried equipment
- Compliance status by category
- Predicted failures (next 30/60/90 days)

Visualizations:

- Failure timeline (interactive)
- Equipment dependency graph
- Maintenance vs. actual failure distribution
- Cost impact analysis

4.3 Admin Panel

- Document upload & preprocessing pipeline
- Equipment registry management
- User access controls
- System health & vector DB stats

5. Technology Stack & Architecture

5.1 Backend

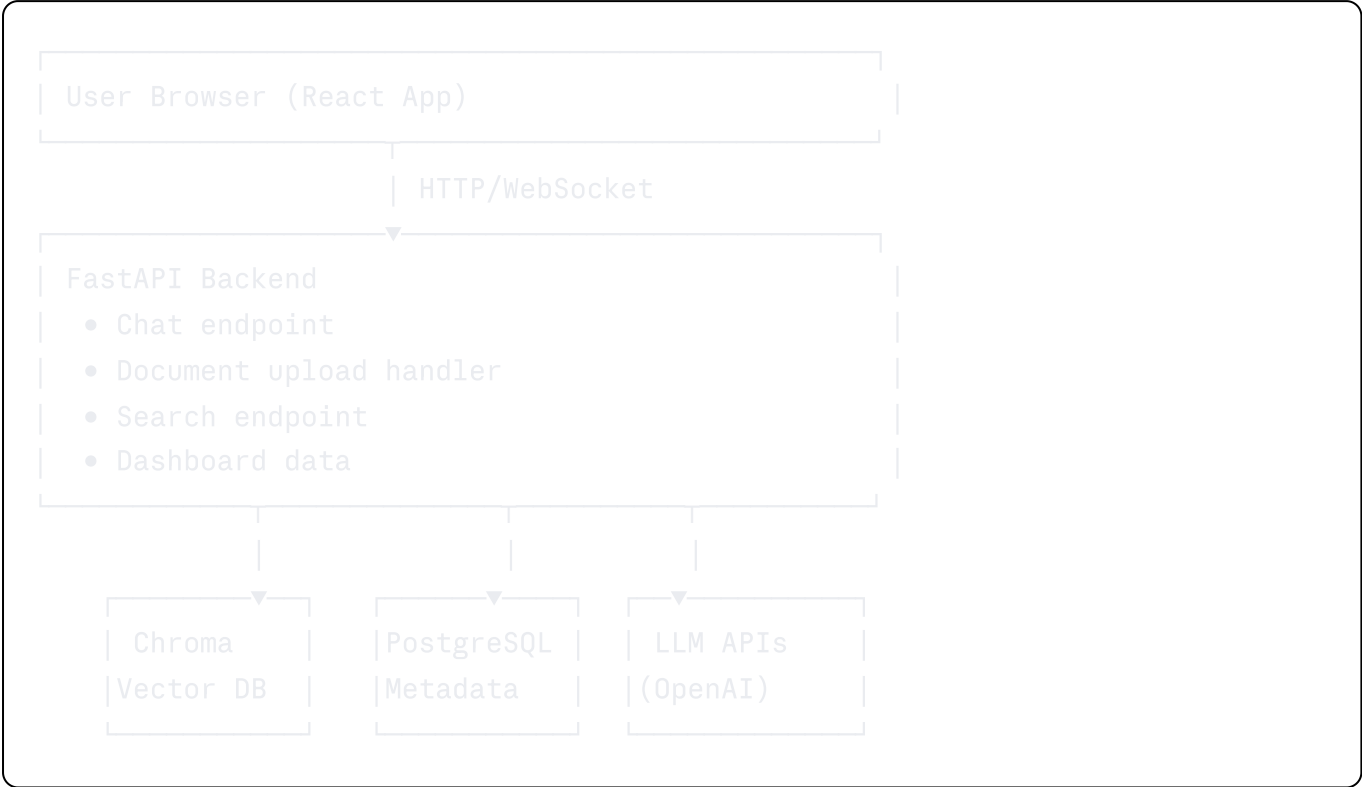
Component	Technology	Rationale
API Framework	FastAPI	Async, high performance, built-in validation
LLM Provider	OpenAI GPT-4 (primary) / Gemini (fallback)	Best instruction following + reasoning
Vector Database	Chroma (easy) or Pinecone (scalable)	Both work; Chroma deployable locally
Text Embeddings	OpenAI <code>text-embedding-3-small</code>	Production-grade, affordable
OCR	Tesseract + EasyOCR	Local deployment, no API costs
PDF Processing	PyPDF2 + pdfplumber	Extract text + table structure
Database	PostgreSQL	Store metadata, user queries, citations
Document Storage	Local filesystem / S3	Store uploaded docs for reference

5.2 Frontend

Component	Technology	Rationale
UI Framework	React (preferred) or Streamlit (rapid prototyping)	React for hackathon presentation; Streamlit if time-constrained
Styling	Tailwind CSS	Fast, clean UI
Real-time Updates	WebSockets (FastAPI)	Live alerts, query status

Component	Technology	Rationale
Charting	Recharts / D3.js	Dashboard visualizations

5.3 Infrastructure



5.4 Data Pipeline



6. Implementation Approach

Phase 1: Foundation (Hours 0-8)

Goal: Working chat with basic RAG

1. Setup

- FastAPI skeleton + routes
- Streamlit/React frontend skeleton
- Chroma local instance
- PostgreSQL local instance

2. Document Pipeline

- PDF text extraction
- Simple chunking (500 tokens + overlap)
- Embedding generation + storage

3. Basic Search

- Vector similarity search
- Return top-3 chunks

4. LLM Integration

- Connect to OpenAI GPT-4
- Prompt: "Answer based on provided documents"
- Add citations

5. Simple UI

- Text input → get answer
- Show source citations

Phase 2: Intelligence (Hours 8-16)

Goal: Add pattern detection & smart features

1. Root Cause Analysis

- Extract failure types from documents
- Categorize by equipment + cause
- LLM-powered analysis: "Why this pattern?"

2. Temporal Features

- Extract dates from documents
- Build failure timeline
- Calculate frequency, trends

3. Equipment Registry

- Extract equipment mentions
- Build name-to-ID mapping
- Link documents to equipment

4. Smart Search Ranking

- Implement BM25 hybrid search

- Boost recent documents
- Boost critical incidents

5. Dashboard MVP

- Equipment list with failure count
- Recent alerts
- Document upload status

Phase 3: Polish & UX (Hours 16-20)

Goal: Make it production-ready

1. Frontend Improvements

- Equipment search/filters
- Timeline visualization
- Cost analysis breakdown

2. Error Handling

- Invalid PDFs
- API failures
- Graceful degradation

3. Demo Data

- Create sample maintenance reports
- Add failure scenarios
- Pre-load equipment database

4. Presentation

- Live demo walkthrough
- Explain architecture
- Show differentiators

7. Stretch Features (If Time Allows)

Priority 1 (Highest Impact):

- ☐ **Equipment relationship graph** (impact analysis)
- ☐ **Predictive failure alerts** (anomaly detection)
- ☐ **Cost-benefit analysis** for recommendations

Priority 2:

- ☐ **Table extraction** from PDFs (better specs handling)
- ☐ **Compliance checker** (automatic violation detection)

- ☐ **Export reports** (PDF summaries)

Priority 3:

- ☐ **Multi-language support** (global companies)
 - ☐ **User feedback loop** (improve answers over time)
 - ☐ **Integration with maintenance tracking systems** (read-only API)
-

8. Why This Stands Out

vs. Basic RAG Chatbots:

❌ Generic RAG: "Here's the document chunk" ✅ **Your solution:** "Here's why, when it's likely to happen again, and what it costs"

vs. General Knowledge Systems:

❌ Missing domain knowledge ✅ **Yours:** Equipment-aware, temporal, predictive

vs. Database Query Tools:

❌ Rigid schemas, slow to set up ✅ **Yours:** Flexible document upload, natural language queries

9. Success Metrics for Judges

Technical Excellence:

- Hybrid search implementation (not naive)
- Temporal/predictive features
- Clean architecture

Business Impact:

- Clear ROI calculation
- Real industrial problems
- Scalable approach

Execution:

- Working demo
- Handles edge cases (scanned docs, poor quality)
- Code clarity

Innovation:

- Root cause analysis engine (your differentiator)
 - Equipment relationship mapping
 - Cost-benefit analysis
-

10. Sample Demo Scenario

Setup: Pre-load 10-15 maintenance reports for a fictional manufacturing plant

Demo Flow:

- Engineer Query:** "Pump A has been failing too often. What's going on?"
 - AI finds 7 failures in 18 months
 - Identifies bearing overheating as root cause
 - Shows 3 similar cases
 - **Result:** "Replace cooling system + extend maintenance schedule → Save \$50K/year"
 - Manager Query:** "Which equipment needs urgent attention?"
 - AI flags 2 items due for inspection
 - Predicts Compressor-7 likely to fail in 30 days
 - Shows cost impact of downtime
 - **Result:** Dashboard with ranked recommendations
 - Quality Check:** "Is this equipment compliant?"
 - Finds 1 violation (overdue inspection)
 - **Result:** Auto-generates compliance report
-

11. Competitive Advantages Summary

Feature	Standard RAG	Your Solution
Search	Text similarity	Equipment-aware + temporal
Answers	Document chunks	Analyzed root causes
Predictions	None	Failure prediction
Citations	Generic	Specific + ranked by relevance
Context	Single query	Conversation + equipment memory
Actionability	Information only	Recommended actions + ROI

Feature	Standard RAG	Your Solution
Scalability	Linear	Relationship graph (exponential insights)

12. Risk Mitigation

Risk	Mitigation
OCR fails on poor-quality scans	Use EasyOCR + Tesseract ensemble; manual review option
Embedding quality issues	Use proven OpenAI embeddings; not custom models
LLM hallucination	Strict instruction: "Only answer from provided docs"; show confidence scores
Performance on large doc sets	Chroma indexing scales to 1M+ vectors; implement pagination
Demo data unrealistic	Use anonymized real-world maintenance data (publicly available)

13. Deployment Strategy

Hackathon Demo:

```
bash

# Local deployment
python -m uvicorn backend:app --reload
streamlit run frontend.py # or: npm start (React)
```

Post-Hackathon Production:

- Deploy backend to AWS/GCP
- Frontend to Vercel/Netlify
- Vector DB to Pinecone
- Documents to S3
- PostgreSQL to managed RDS

14. Timeline & Deliverables

Milestone	Time	Deliverable
Phase 1	8 hrs	Working chat + basic RAG
Phase 2	8 hrs	Root cause analysis + timeline + dashboard
Phase 3	4 hrs	Polish + live demo
Total	20 hrs	Production-grade MVP

15. Team Skills Alignment

Your background is **perfect** for this:

- ✔ ML/NLP experience (BiLSTM, computer vision fundamentals)
- ✔ Python proficiency
- ✔ Data processing (ETL, SQL)
- ✔ n8n automation (document workflow integration)
- ✔ API integration experience

Action: Leverage these in your pitch.

16. Final Pitch Message

"While everyone builds generic chatbots over PDFs, we're building an *expert system* that understands *why* equipment fails and *predicts* failures before they happen. Our system doesn't just retrieve documents—it analyzes patterns, calculates ROI, and recommends actions. For manufacturers losing millions to preventable downtime, this is not a nice-to-have; it's critical infrastructure."

Document Version: 1.0
Last Updated: July 2026
Status: Ready for Hackathon Submission